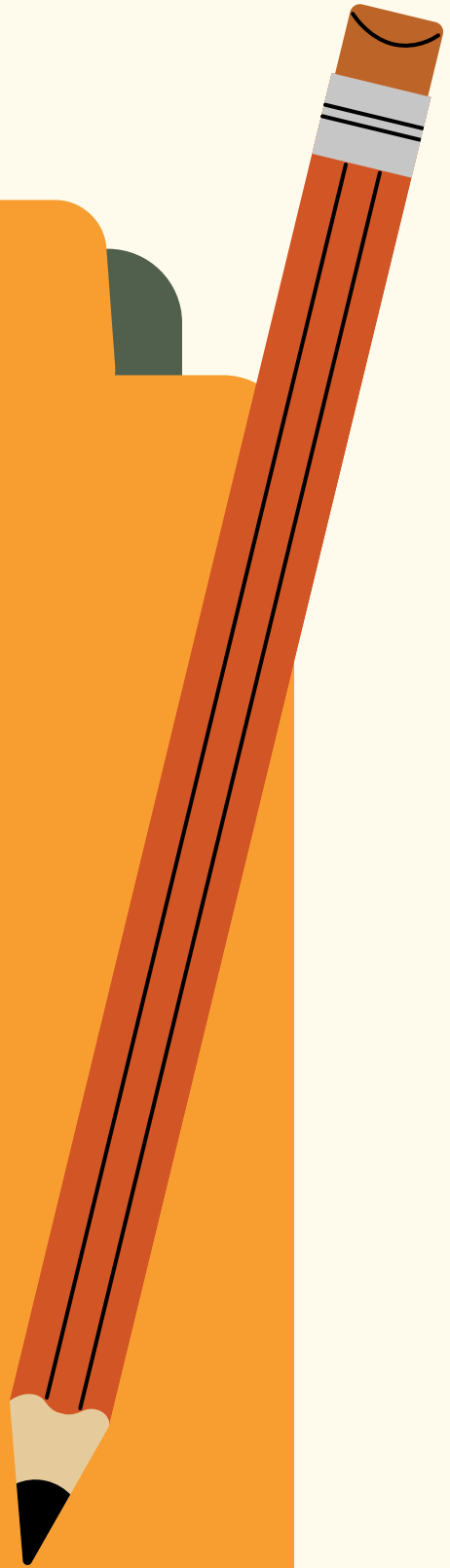
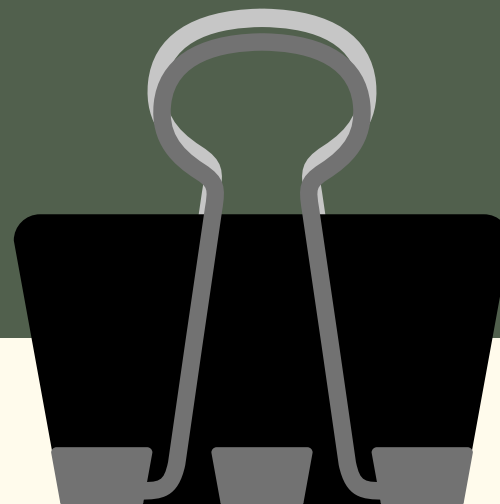


From numbers to AI

PROGRAMMING

파이썬





질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



Function

함수



함수



- 함수로 원하는 기능을 구현
- 내장 함수와 외장 함수를 활용하고, 원하는 걸 찾아 쓸 수 있게

what_is_function

코드를 재사용하고, 가독성을 높임

Function of Python

함수란?

- 입력을 받아 정해진 동작(기능)을 수행해 결과를 출력
- Ex) 화씨 온도를 섭씨 온도로 바꾸는 코드
입력: 화씨 온도/ 동작: 화씨 섭씨 변환/ 출력: 섭씨 온도
→ 이 때 동작을 정의 하고 이름붙이는 것

```
f = int(input("화씨온도"))  
c = (f-32)*5/9  
print(f"섭씨온도: {c}")
```

```
def fa_2_ce(f):  
    return (f-32)*5/9  
  
fa = int(input("화씨온도: "))  
print(f"섭씨온도: {fa_2_ce(fa)}")
```

Function of Python

왜 함수를 쓰는가?

```
f = int(input("화씨온도"))  
c = (f-32)*5/9  
print(f"섭씨온도: {c}")
```

```
f = int(input("화씨온도"))  
c = (f-32)*5/9  
print(f"섭씨온도: {c}")
```

⋮

```
f = int(input("화씨온도"))  
c = (f-32)*5/9  
print(f"섭씨온도: {c}")
```

가독성 **떨어짐**, 생산성 **떨어짐**!



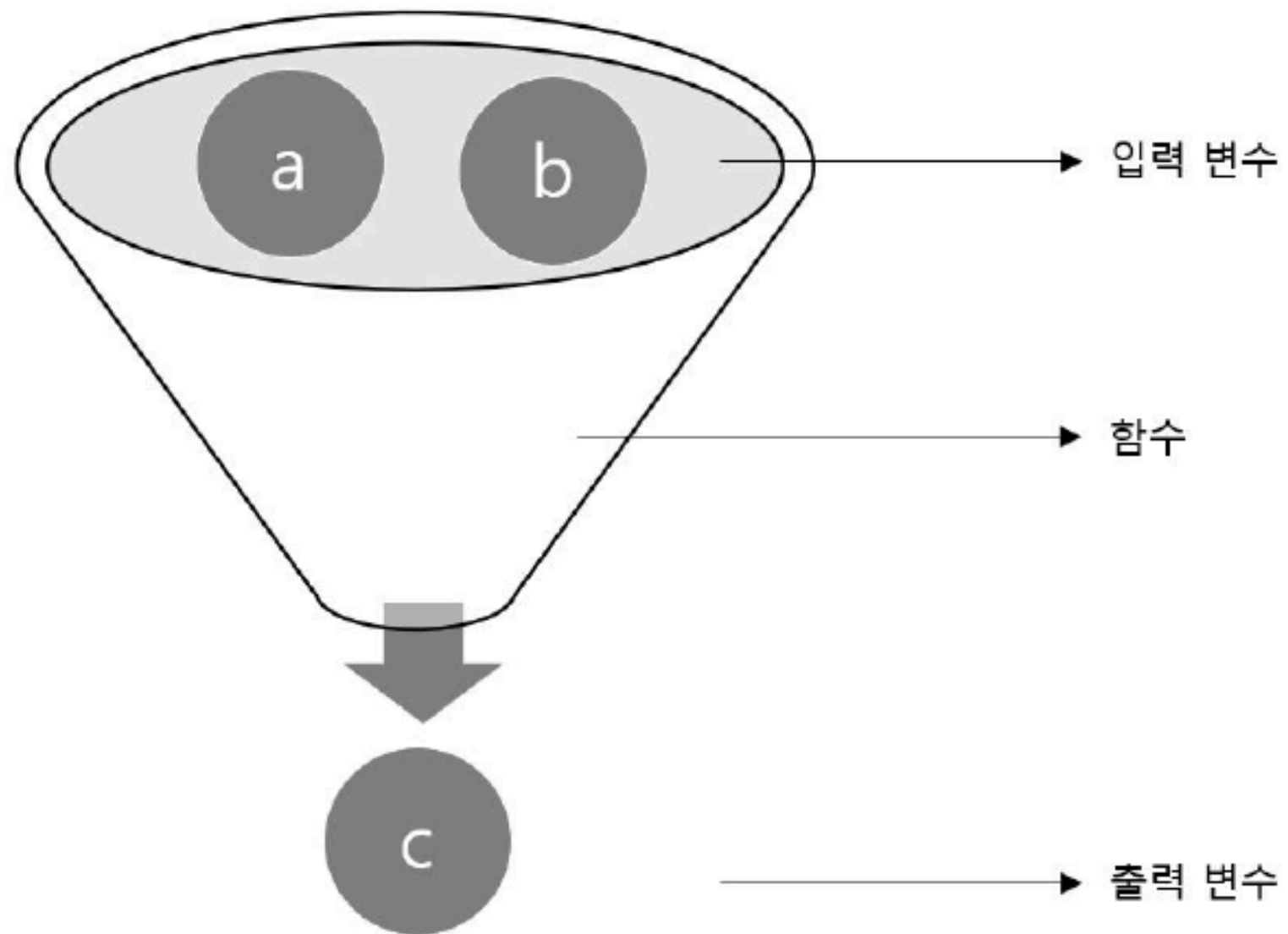
```
def fa_2_ce(f):  
    return (f-32)*5/9  
  
fa = int(input("화씨온도: "))  
print(f"섭씨온도: {fa_2_ce(fa)}")
```

여러 번 사용할 기능을 **패키징**

“매번 온도 변환할 때마다 식을 써줄 것이냐,
만들어 둔 함수에 값만 넣을 것이냐”

Function of Python

함수란?



함수 이름 ← → 입력 변수

```
def sum(a,b):  
  
    c = a + b  
  
    return c
```

출력 변수

Function of Python

함수 만들기

```
def func():  
    --- code ---  
    return X
```

- def → 예약어 함수 정의가 시작될 것임을 나타낸다.
- func → 함수 이름 (snake_case: 함수명으로 영어(소문자 권장), 숫자, _(underscore)의 조합으로 작성한다)
- () → 괄호 사이에 입력변수(매개변수)들을 넣을 수 있다. 아무것도 넣지 않아도 상관없다.
- : → 함수명 정의가 끝났음
- return → 함수가 반환할것을 정하고 종료.

Function of Python

함수 만들기

- 인자의 값을 정의하는 두 가지 방법

위치 인자: 각 인자를 순서대로 매개 변수와 연결

```
def my_add_with_default(a, b, c = 3):  
    print(f"a: {a}, b: {b}, c: {c}")  
    return a + b + c, a - b - c
```

```
print(my_add_with_default(1, 2, 4))
```

```
a: 1, b: 2, c: 4  
(7, -5)
```

키워드 인자: 각 매개변수의 값을 직접 지정

```
def my_add_with_default(a, b, c = 3):  
    print(f"a: {a}, b: {b}, c: {c}")  
    return a + b + c, a - b - c
```

```
print(my_add_with_default(c=4, a=1, b=2))
```

```
a: 1, b: 2, c: 4  
(7, -5)
```

```
print(my_add_with_default(a=1, 2))  
# print(my_add_with_default(c=1, 2, 4)) # error!
```

```
File "<ipython-input-7-4d3410f2c80a>", line 8
```

```
print(my_add_with_default(a=1, 2))
```

```
SyntaxError: positional argument follows keyword argument
```

(단, 두 가지 방법을 같이 쓸 수 없다)

Function of Python

함수 만들기

- 이름은 보기만 해도 어떤 함수 혹은 변수인지를 알 수 있게 짓는 게 좋다
- 변수에 직접 값을 넣어줄 수도 있다

```
def my_add(a, b):  
    return a + b  
  
result_ = my_add(a=3, b=7)  
print(result_)
```

10

```
def my_add(a, b):  
    return a + b
```

```
a = 3  
b = 4  
c = my_add(a, b)  
print(c)
```

7

Function of Python

함수 만들기

- 매개변수의 기본 값을 정할 수 있다
 - 함수를 정의할 때 매개변수의 기본 값을 정의
(가장 뒤에 위치한 매개변수부터 차례대로 정의)
 - 인자로 해당 매개변수의 값을 주지 않으면 기본 값을 사용

(1+2+3, 1-2-3)

(1+2+4, 1-2-4)

```
def dollar_to_won(dollar, dwrate = 1200, bank_ggulgguk = 0.5):  
    my_won = dollar * dwrate * (1 - bank_ggulgguk / 100)  
    return f"{dollar}$ = {my_won}원"  
  
dollar_to_won(100)  
  
'100$ = 119400.0원'
```

```
def my_add_with_default(a, b, c = 3):  
    return a + b + c, a - b - c  
  
print(my_add_with_default(1, 2))  
print(my_add_with_default(1, 2, 4))  
  
(6, -4)  
(7, -5)
```

Function of Python

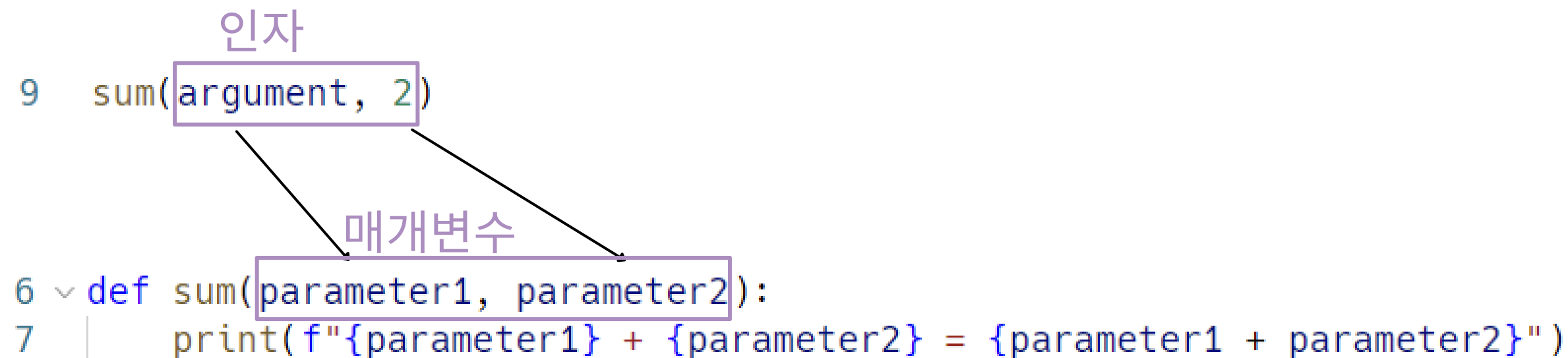
매개변수와 인자

```
argument = 2
```

```
def sum(parameter1, parameter2):  
    print(f"{parameter1} + {parameter2} = {parameter1 + parameter2}")
```

```
sum(argument, 2)
```

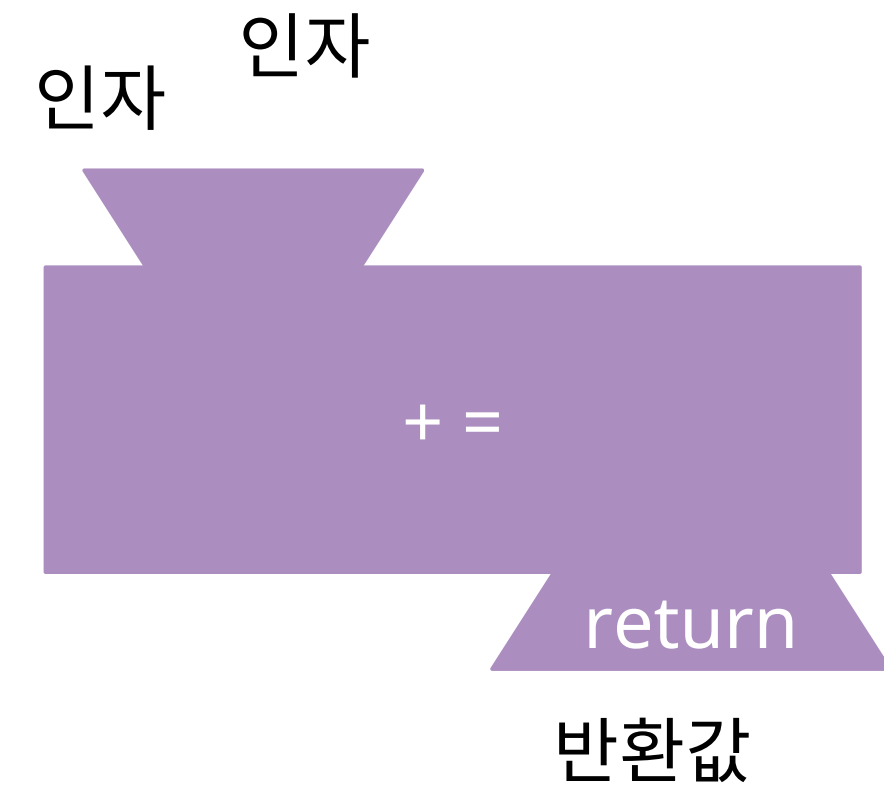
- 함수는 인자를 받아 매개변수에 전달을 해주고, 함수 안에서 매개변수를 사용합니다.



Function of Python

return

- 함수가 끝나면 결과값을 반환할 수 있다.
- return 예약어로 끝나지 않더라도, 자동적으로 None을 반환함
- 반환값을 받아서 변수에 담지 않아도 괜찮다.
- Return가 없어도 함수는 에러 없이 돌아가고, None을 반환했다



```
def return_none():  
    print("dd")  
  
print(return_none())  
  
dd  
None
```

Function of Python

return

- return을 하면 함수가 즉각 종료된다.
- 예시: for문이 처음 한 번만 돌고 끝나버렸다
- 이러한 성질을 활용해 반복문을 탈출하는 용도로 return을 사용할 수도 있다.

```
def star_stack(n):  
    for i in range(1, n + 1):  
        print("*" * i)
```

```
star_stack(5)
```

```
*  
**  
***  
****  
*****
```

```
def star_stack(n):  
    for i in range(1, n + 1):  
        return "*" * i
```

```
star_stack(5)
```

```
'*'
```

Function of Python

return

```
def test_return():  
    a = 0  
    while 1:  
        print(f"a={a}")  
        a+=1  
        b = 0  
        while 1:  
            print(f"b={b}")  
            b+=1  
            if b>3:  
                return  
test_return()
```

a=0
b=0
b=1
b=2
b=3

```
is_break = False
```

```
a = 0
```

```
while 1:  
    print(f"a={a}")  
    a+=1  
    b = 0  
    while 1:  
        print(f"b={b}")  
        b+=1  
        if b>3:  
            is_break = True  
            break  
    if is_break:  
        break
```

a=0
b=0
b=1
b=2
b=3

하나의 반복만 탈출!

Function of Python

함수 호출

함수를 만들면 들여쓰기 후 코드를 작성하면 됩니다.

함수명() 로 함수를 호출 할 수 있습니다.

함수는 정의 전에 호출 할 수 없습니다.

```
func()  
  
def func():  
    pass
```

```
def is_func():  
    print("함수입니다")  
  
print("시작")  
is_func()  
print("끝")
```

```
시작  
함수입니다  
끝
```

Function of Python

함수 만들기

- 익숙한 구구단을 함수로
 - 구구단 n단이 필요할 때마다 gugudan(n) 한 줄 칠 것인가 반복문을 일일이 써줄 것인가
 - 구구단을 2번 정도 입력할 것이라면 그냥 반복문을 쳐라, 하지만 끝도 없이 써야 한다면?

```
# 구구단 no 함수 버전

# 구구단 출력
for i in range(1, 10):
    print(f'{2} x {i} = {2 * i}')
for i in range(1, 10):
    print(f'{3} x {i} = {3 * i}')

# ...
for i in range(1, 10):
    print(f'{9} x {i} = {9 * i}')
```

```
# 구구단 함수 버전
def gugudan(num):
    for i in range(1, 10):
        print(f'{num} x {i} = {num * i}')

# 구구단 출력
gugudan(2)
gugudan(3)
# ...
gugudan(9)
```

Function of Python

함수 만들기

- print 결과를 한 칸 띄우고 출력하고 싶다면?

```
for i in range(10):  
    print(i, end=' ')
```

0 1 2 3 4 5 6 7 8 9

```
out_str = ""  
for i in range(10):  
    out_str += str(i) + ' '  
print(out_str)
```

0 1 2 3 4 5 6 7 8 9

```
def str_spacer(input_str):  
    try:  
        out_str = ""  
        for single_char in input_str:  
            out_str += single_char + ' '  
        return out_str[:-1]  
    except:  
        print('input must be string')  
        return None
```

```
spaced_out = str_spacer('12345')  
print(spaced_out)  
print(len(spaced_out))
```

1 2 3 4 5
9

Function of Python

함수 만들기

- print 결과를 한 칸 띄우고 출력하고 싶다면?
 - 원래는?
 - print도 함수! 여러 매개 변수가 있고 매개 변수 end는 '\n'가 default값으로 설정

```
help(print)
```

```
Help on built-in function print in module builtins:
```

```
print(...)
```

```
    print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

```
Prints the values to a stream, or to sys.stdout by default.
```

```
Optional keyword arguments:
```

```
file: a file-like object (stream); defaults to the current sys.stdout.
```

```
sep: string inserted between values, default a space.
```

```
end: string appended after the last value, default a newline.
```

```
flush: whether to forcibly flush the stream.
```

Function of Python

함수 만들기

- 타입 힌트: 뭘 넣어야 하고 뭐가 나오는지
- 예시: 입력도 출력도 str
- 강제성은 없다
- 타입 힌트 확인 가능

```
def type_hint(input_: str)->str:  
    return 'okay to put different types of variables in it'
```

```
for_real = type_hint(123)  
print(for_real)
```

```
type_hint.__annotations__
```

```
okay to put different types of variables in it  
{'input_': str, 'return': str}
```

```
def str_spacer(input_str:str)->str:  
    try:  
        out_str = ""  
        for single_char in input_str:  
            out_str += single_char + ' '  
        return out_str[:-1]  
    except:  
        print('input must be string')  
        return None
```

```
spaced_out = str_spacer('12345')  
print(spaced_out)  
print(len(spaced_out))
```

```
1 2 3 4 5  
9
```

Function of Python

함수 만들기

- 함수는 여러 개의 값을 반환 할 수 있다?
 - 실제로는 하나의 tuple 로 return

```
def my_add_with_default(a, b, c = 3):  
    return a + b + c, a - b - c  
  
print(my_add_with_default(1, 2))  
print(my_add_with_default(1, 2, 4))  
print(my_add_with_default(c=1, a=2, b=4))
```

```
(6, -4)  
(7, -5)  
(7, -3)
```

```
print(type(my_add_with_default(1, 2)))  
  
<class 'tuple'>
```

Function of Python

함수 만들기

- *를 이용해 여러 개 받을 인자를 선택할 수도 있다
 - 위치 인자가 *를 만나면 그 이후의 위치 인자들은 모두 하나의 tuple로 관리
- 키워드 인자의 경우 *가 아닌 **을 통해 packing

```
def keyword_packing(**kwargs):  
    print(kwargs)  
    print(type(kwargs))  
  
keyword_packing(name = "CM", age = 7)  
  
{'name': 'CM', 'age': 7}  
<class 'dict'>
```

```
def menu_teller(rice, main_dish, *side_dishes):  
    print(f"밥: {rice}")  
    print(f"메인: {main_dish}")  
    if side_dishes:  
        print("---반찬---")  
        for side in side_dishes:  
            print(side)  
  
menu_teller("현미밥", "참치밥전", "김치", "마늘장아찌")
```

```
밥: 현미밥  
메인: 참치밥전  
---반찬---  
김치  
마늘장아찌
```

Function of Python

함수 만들기

- 하나의 매개변수에 대응하는 인자를 여러 개 쓸 수 있음 (packing)
 - tuple, set, list, dict 를 **unpacking** 하여 넣을 수도 있음
- 매개 변수 앞에 * 또는 **를 붙여서 지시
 - *을 쓰면 tuple 로 packing 되고 **을 쓰면 dict 로 packing 된다

```
def sum_(*nums):  
    result_ = 0  
    for num in nums:  
        result_ += num  
    return result_  
  
print(sum_(1, 2, 3, 4, 5))
```

15

```
def keyword_packing(**kwargs):  
    print(kwargs)  
    print(type(kwargs))  
  
keyword_packing(name = "CM", age = 7)  
  
{'name': 'CM', 'age': 7}  
<class 'dict'>
```

Function of Python

함수 만들기

- Unpacking
 - list, set, tuple, dict 타입의 객체를 해체하여 모든 원소를 꺼냄 (해당 타입의 **인자** 앞에 *을 붙임)
 - dict 객체의 경우 unpacking 하여
 - *가 붙은 매개변수에 인자로 넣으면 key만 packing 되고
 - **가 붙은 매개변수에 인자로 넣으면 다시 dict 로 packing 된다

```
a = [1, 2, 3]
print(sum_(*a))
print(sum_(*[1, 2, 4]))
print(sum_(*{1, 2, 5}))
```

6
7
8

```
def keyword_packing(**kwargs):
    print(kwargs)
    print(type(kwargs))

keyword_packing(name = "CM", age = 7)
```

{'name': 'CM', 'age': 7}
<class 'dict'>

```
d = {1: 'a', 2: 'b'}
print(sum_(*d))
```

3

Function of Python

함수 만들기

- 일반 매개변수, * 가 붙은 인자, **가 붙은 인자를 같이 쓸 때는 순서대로 써야한다.

```
def menu_teller(rice, main_dish, *side_dishes):  
    print(f"밥: {rice}")  
    print(f"메인: {main_dish}")  
    if side_dishes:  
        print("---반찬---")  
        for side in side_dishes:  
            print(side)  
  
menu_teller("현미밥", "참치밥전", "김치", "마늘장아찌")
```

```
밥: 현미밥  
메인: 참치밥전  
---반찬---  
김치  
마늘장아찌
```

```
def menu_teller(*side_dishes, rice, main_dish):  
    print(f"밥: {rice}")  
    print(f"메인: {main_dish}")  
    if side_dishes:  
        print("---반찬---")  
        for side in side_dishes:  
            print(side)  
  
menu_teller("현미밥", "참치밥전", "김치", "마늘장아찌")
```

TypeError Traceback (most recent call last)

<ipython-input-8-caf495c4c104> in <module>()

7 print(side)

8

----> 9 menu_teller("현미밥", "참치밥전", "김치", "마늘장아찌")

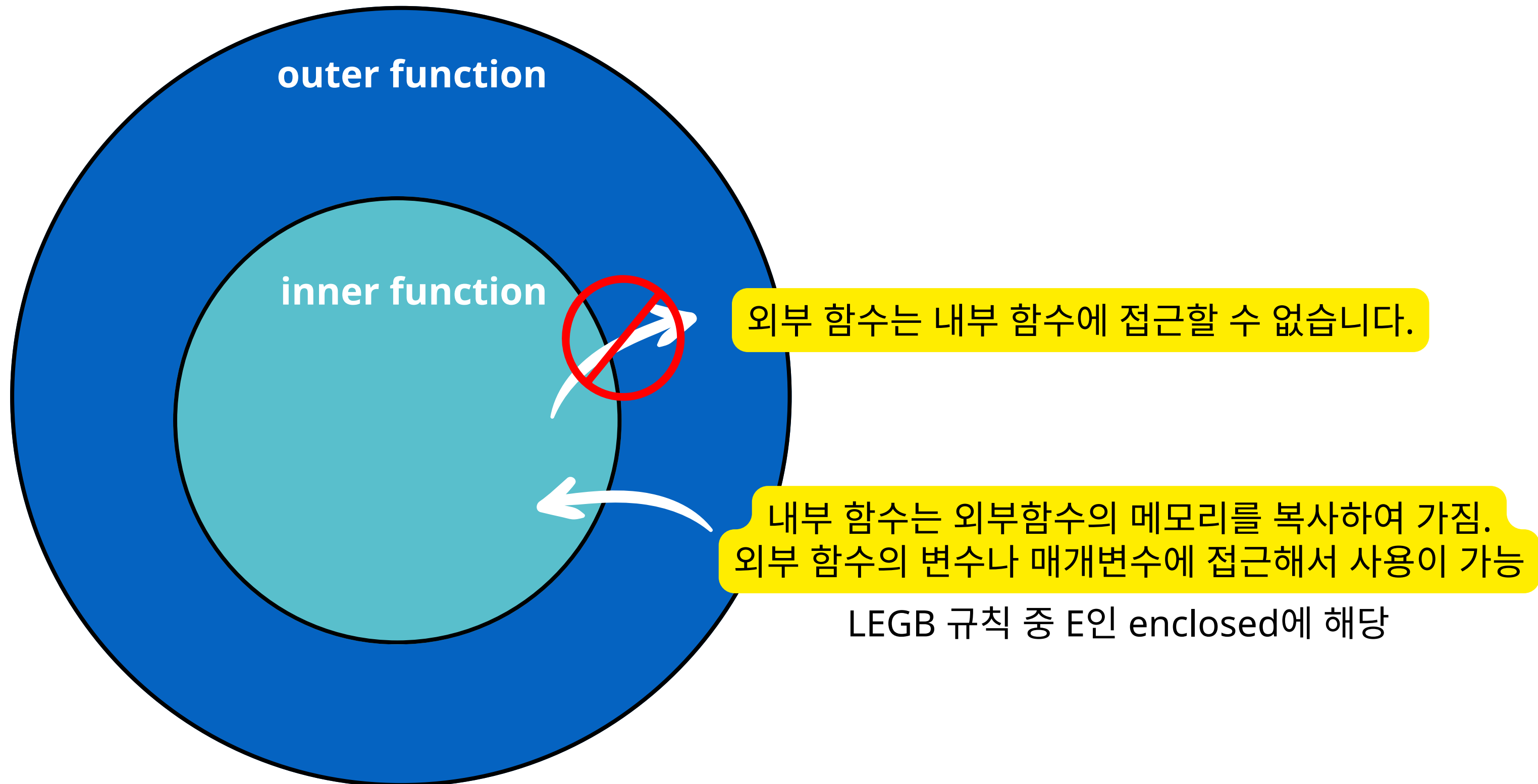
TypeError: menu_teller() missing 2 required keyword-only arguments: 'rice' and 'mai'

중첩함수와 클로저

first-class citizen

nested function

중첩함수



nested function

중첩함수

정의:

- 함수 내부에 정의된 함수
- 바깥 함수의 변수에 접근 가능

특징:

- 바깥 함수의 지역 변수 사용 가능
- 보통 클로저 생성의 기반이 됨

```
def outer():  
    x = 10  
    def inner():  
        print(f"x는 {x}입니다.")  
    inner()
```

x는 10입니다.

inner() 함수는 outer()의 지역 변수 x에 접근할 수 있음

closure

클로저

정의:

- 바깥 함수의 변수를 참조하는 내부 함수
- 바깥 함수가 종료된 이후에도 변수에 접근 가능

조건:

- 중첩 함수일 것
- 바깥 함수의 변수를 참조할 것
- 바깥 함수가 종료되어도 내부 함수가 반환되어 존재할 것

outer()는 종료되었지만 x = 10은 inner() 안에서 살아있음
inner()는 x에 대한 참조를 유지

```
def outer():  
    x = 10  
    def inner():  
        return x + 5  
    return inner # 함수 객체 반환
```

```
f = outer()  
print(f()) # inner() 호출
```

15

closure

클로저

```
def make_counter():  
    count = 0  
    def counter():  
        nonlocal count  
        count += 1  
        return count  
    return counter
```

```
c1 = make_counter()  
print(c1()) # 1  
print(c1()) # 2
```

1

make_counter() 실행 후 count는 계속 유지됨

2

함수가 상태를 기억하는 상태 유지 함수(stateful function)로 활용됨

장점:

- 상태 유지 가능
- 캡슐화 효과
- 콜백 함수, 데코레이터 구현에 자주 사용

주의점:

- 너무 많은 상태 유지 시 복잡도 증가
- 메모리 누수 위험 (참조 순환)

```
def calculator(x):
    def operate(y, op='+'):
        if op == '+':
            return x + y
        elif op == '-':
            return x - y
        elif op == '*':
            return x * y
        elif op == '/':
            return x / y
        else:
            return "지원하지 않는 연산입니다."
    return operate

f = calculator(10)
print(f(5))          # 기본값 '+' => 15
print(f(3, '*'))     # 10 * 3 => 30
print(f(2, '-'))     # 10 - 2 => 8
print(f(4, '/'))     # 10 / 4 => 2.5
```

```
def power_base(base):
    def raise_to_power(exp):
        return base ** exp
    return raise_to_power
```

```
f = power_base(3)    # base = 3
print(f(2))          # 3^2 = 9
print(f(4))          # 3^4 = 81
```

```
square = power_base(2)
cube = power_base(3)
```

```
print(square(5))     # 2^5 = 32
print(cube(2))        # 3^2 = 9
```

```
def make_coupon(discount_rate):
    def apply_discount(price):
        return price * (1 - discount_rate)
    return apply_discount
```

```
coupon10 = make_coupon(0.10)    # 10% 할인 쿠폰
coupon25 = make_coupon(0.25)    # 25% 할인 쿠폰
```

```
print(coupon10(10000))    # 10000원 상품에 10% 할인 → 9000
print(coupon25(10000))    # 10000원 상품에 25% 할인 → 7500
```

```
def outer():
    outer_var = 10

    def inner(x):
        return outer_var ** x

    return inner

f = dir(outer())
print(f)
```

```
['__annotations__', '__call__', '__class__', '__closure__', '__code__',
 '__defaults__', '__delattr__', '__dict__', '__dir__', '__doc__',
 '__eq__', '__format__', '__ge__', '__get__', '__getattr__',
 '__globals__', '__gt__', '__hash__', '__init__', '__init_subclass__',
 '__kwdefaults__', '__le__', '__lt__', '__module__', '__name__',
 '__ne__', '__new__', '__qualname__', '__reduce__', '__reduce_ex__',
 '__repr__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__']
```

```
[105] 1 print(f.__closure__)
      2 print(f.__closure__[0].cell_contents)
```

```
⇒ (<cell at 0x7b785e4b4310: int object at 0xa40ca8>,)
10
```

```
[108] 1 f = outer()
      2 print(f.__code__)
```

```
⇒ <code object inner at 0x7b79affabd20, file "<ipython-input-102-8c252cff067e>", line 4>
```

```
[109] 1 print("co_varnames      :", f.__code__.co_varnames)
      2 print("co_argcount      :", f.__code__.co_argcount)
      3 print("co_freevars      :", f.__code__.co_freevars)
      4 print("co_cellvars      :", f.__code__.co_cellvars)
```

```
⇒ co_varnames      : ('x',)
   co_argcount      : 1
   co_freevars      : ('outer_var',)
   co_cellvars      : ()
```

f.__code__.co_freevars → 클로저에서 외부 스코프에서 가져온 변수들
f.__closure__ → 그 변수들의 실제 값
f.__code__.co_varnames → 함수 내부 변수 목록

클로저 왜 쓰는가!!!

- global 변수를 사용하고 싶지 않을 때
 - global 변수를 사용하지 않고, nonlocal 변수를 사용하여 유사한 동작을 구현할 수 있습니다. 또한 nonlocal 변수를 사용하게 되면 데이터를 숨길 수 있기 때문에 일부 정보를 은닉할 수 있게 됩니다.
- 클래스를 사용하지 않기 위해
 - 경우에 따라 모든 로직에 대해 클래스화 하여 멤버 함수를 만들어서 사용하는 것이 비효율적일 수 있습니다. 다루는 변수와 함수가 많지 않은 경우, 클로저로 구현하는 것이 효율적일 수 있습니다.
- 데코레이터(Decorator)를 사용하기 위해

binding

LEGB 규칙

Function of Python

전역변수

- 어디서든 호출이 가능한 변수

```
1  a = "global" 메인코드에 작성하면 전역 변수
2
3  ✓ def func():
4    |   print(a) 함수안에서도 호출 가능
```

Function of Python

지역변수

- 특정 범위에서만 호출이 가능한 변수

```
def func():  
    a = 10  
    print(a)
```

함수 안에서 정의하고 호출하는 변수

```
func()  
print(a)
```

함수 밖에서 호출하려고 하면 오류 발생

함수 안에서 정의한 변수는 함수 안에서만 사용이 가능하고, 밖으로 나오면 변수가 사라진다.

Function of Python

특징

```
1 def func():  
2     a = 10  
3     print(a)
```

함수 안에서 변수를 호출 할 때 변수명이 같다면 지역 변수를 우선적으로 호출함

```
4  
5 a = 20  
6 func()
```

```
7 print(a)
```

함수 밖으로 나오면 지역변수 a는 사라졌으므로 전역변수 a를 호출

함수 안에서 똑같은 변수명을 호출하면 지역변수를 우선적으로 선택
(매개변수도 지역변수!)

Function of Python

전역 변수 vs 지역 변수

전역 변수

코드의 모든 곳(전역)에서
접근 할 수 있다.

지역 변수

코드의 일부분(지역)에서
접근 할 수 있다.
지역을 벗어나면 사라진다.

변수의 생존 범위를 알지 못하면, 오류 발생

Function of Python

Global 키워드

```
1  def func():
2      a += 10
3      print(a)
4
5  a = 10
6  func()
```

전역 변수를 함수 안에서 변경하려고 하면
오류 발생

예외가 발생했습니다. **UnboundLocalError** ×
local variable 'a' referenced before assignment

지역변수 'a' 가 정의전에 사용되었습니다.

함수 안에서 사용하는 변수가 지역변수가 아닌 전역 변수라는 것을 알려주어야 한다.

Function of Python

Global 키워드

```
1  def func():
2      global a
3      a += 10
4      print(a)
5
6  a = 10
7  func()
8  print(a)
```

함수 안에서 a는 전역변수 a를 사용한다는 선언

전역 변수를 사용했기 때문에 함수안에서 변경된 값이 반영된다.

출력

20
20

Function of Python

스코프

- 변수의 접근 범위
- 함수의 안에서 정의된 변수는 어디까지 쓸 수 있을까?
- 전역 변수와 지역 변수 - 네임스페이스

```
def func():  
    a = 10
```

```
func()  
print(a)
```

```
graph BT; Local[Local] --> Enclosed[Enclosed]; Enclosed --> Global[Global]; Global --> Built-in[Built-in];
```

[Python] Scoping Rule의 종류

Scope개념은 크게 4종류로 나뉘 볼 수 있다. Local Scope Enclosed Scope Global Scope Built-in Scope 간단하게 다음 문제들을 풀어보면서 자신이 파이썬의 LEGB 룰에 대하여 잘 알고 있는지 확인해보자.

HandSpeed / Jan 20, 2021

안에서는 밖을 볼 수 있다

밖에서 안을 볼 수 없다

순서	이름 공간 (Scope)	설명
L	Local	가장 안쪽의 현재 함수 내 지역 범위
E	Enclosing	중첩 함수 바깥쪽 함수의 범위
G	Global	현재 모듈의 전역 범위
B	Built-in	파이썬이 제공하는 내장 이름들 (len , print , 등)

먼저 Local(inner) 범위에서 찾고,
없으면 Enclosing(outer) 범위에서 찾고,
그래도 없으면 Global, 마지막으로 Built-in까지 찾아감

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        x = "local"
        print(x) # 여기서 local이 가장 우선됨

    inner()

outer()
```

local 출력결과

```
x = "global"

def outer():
    x = "enclosing"

    def inner():
        print(x) # 여기서 enclosing

    inner()

outer()
```

enclosing 출력결과

global

```
x = 10

def change_global():
    global x
    x = 20 # 전역 변수 x를 수정

change_global()
print(x)
```

20

- 현재 함수 안에서 전역(global) 변수에 접근 또는 수정할 때 사용
- 기본적으로 함수 내부에서 변수에 값을 할당하면 Local로 간주되므로, 전역 변수로 수정하려면 global을 명시해야 함

nonlocal

```
def outer():  
    count = 0  
  
    def inner():  
        nonlocal count  
        count += 1  
        return count  
  
    return inner
```

```
f = outer()  
print(f()) # 1  
print(f()) # 2
```

1

2

- 중첩함수 내부에서 바깥(Enclosing) 함수의 변수를 수정할 때 사용
- nonlocal 없이는 바깥 변수는 읽기만 가능하고, 쓰면 새로운 지역변수가 생김
- nonlocal은 LEGB의 E(Enclosing) 범위의 바인딩을 명시적으로 수정함

nonlocal이 빠지면, 에러남

```
def outer():  
    count = 0  
  
    def inner():  
        count += 1 #  
        return count  
  
    return inner
```

읽기는 되지만 쓸수 없어!
쓰려면 새 변수로 지정하고 써야지

UnboundLocalError: cannot access local variable 'count' before assignment

1) local scope

```
def local_scope():  
    local_var = 1  
    print(local_var)  
  
local_scope()  
print(local_var)
```

1

NameError

2) Enclosed Scope

```
def enclosing_scope():  
    enclosing_var = 6  
    def inner_func():  
        inner_var = 3  
        print(enclosing_var * inner_var)  
    inner_func()  
    print(inner_var)  
  
enclosing_scope()
```

18

NameError

3-1) Global Scope

```
global_var = 5

def outer_func():
    outer_var = global_var + 2
    print(outer_var)
    def inner_func():
        inner_var = global_var * 9
        print(inner_var)
    inner_func()

outer_func()
print(global_var)
```

7

45

5

3-2)

```
global_var = 5

def outer_func():
    outer_var = global_var + 2
    print(outer_var)
    def inner_func():
        global_var = global_var * 9
        print(global_var)
    inner_func()

outer_func()
print(global_var)
```

7

UnboundLocalError

namespace

`globals(), locals()`

Function of Python

- 네임스페이스와 변수의 범위
 - 변수와 그 값은 dict 의 key와 value의 형태로 네임스페이스에 저장
 - 전역 변수는 'global' 라는 이름의 네임스페이스에 등록
- 'global' 네임스페이스에 있는 모든 변수의 '이름: 값'을 반환하는 함수 'globals'

- {'__name__': '__main__',

#이외 현재 시스템에 관한 전역 변수들

'a': 1}

- a에 1이라는 값이 저장되어 있는 걸 확인할 수 있다.

```
a = 1
print(globals())
```

Function of Python

- 네임스페이스와 변수의 범위
 - 함수는 전역과는 다른 별도의 네임스페이스를 가지고 있다
 - 함수를 선언하고 그 함수 안에 변수들을 선언한 뒤 globals 함수를 사용해 봐도 함수 내용 전체가 namespace에 들어가 있을 뿐

```
def local_check():  
    a = 1  
    b = 'CM'  
  
print(globals())
```

- 실행 후 추가된 걸 확인해 보면...

```
" def local_check():\n a = 1\n b = ' CM ' \n\nprint(globals()) "
```

a와 b가 추가되지 않았다. 함수 전체가 한 덩어리로 들어가 있을 뿐

Function of Python

- 네임스페이스와 변수의 범위
 - locals 함수를 함수 내부에서 실행해 보자

```
def local_check():  
    a = 1  
    b = 'CM'  
    print(locals())
```

```
local_check()
```

```
{'a': 1, 'b': 'CM'}
```

- a라는 변수에 값 1이, b라는 변수에 값 'CM'이 잘 저장되어 있는 걸 볼 수 있다.

decorator

고수처럼 보이는 가장 쉬운 방법

decorator

데코레이터

정의:

- 데코레이터는 기존 함수를 수정하지 않고, 기능을 추가할 수 있는 함수
- 함수를 인자로 받아, 새로운 함수로 반환함
- @decorator_name 문법을 사용해 간결하게 적용 가능

특징:

- @decorator는 사실 decorator(func)을 적용해주는 문법

```
def say_hi():  
    print("Hi!")
```

```
decorated = decorator(say_hi)  
decorated()
```

@없이도 동작함

```
def decorator(func):  
    def wrapper():  
        print("Before function")  
        func()  
        print("After function")  
    return wrapper
```

```
@decorator  
def say_hello():  
    print("Hello!")
```

```
say_hello()
```

```
Before function  
Hello!  
After function
```

1급시민: func을 전달.
@로 함수를 감싸는 것

decorator

데코레이터

```
def decorator(func):  
    def wrapper(*args, **kwargs):  
        print("Start")  
        result = func(*args, **kwargs)  
        print("End")  
        return result  
    return wrapper
```

@decorator

```
def add(x, y):  
    return x + y
```

```
print(add(3, 5))  # 8
```

Start

End

8

```
import time  
  
def timer(func):  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"실행 시간: {end - start:.4f}초")  
        return result  
    return wrapper
```

@timer

```
def slow_function():  
    time.sleep(1)  
    print("Done")
```

```
slow_function()
```

Done

실행 시간: 1.0000초

```
[23] 1 @timer  
      2 @decorator  
      3 def greet():  
      4     print("Hello!")  
      5  
      6 greet()
```

```
⇒ Start  
   Hello!  
   End  
   실행 시간: 0.0003초
```

- 여러 데코레이터를 겹쳐서 사용 가능
- 실행 순서는 아래 → 위로 적용됨

recursive functions

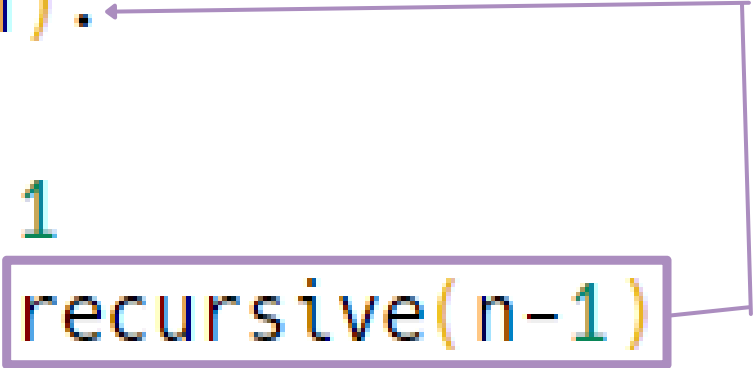
의논할 의

Function of Python

재귀

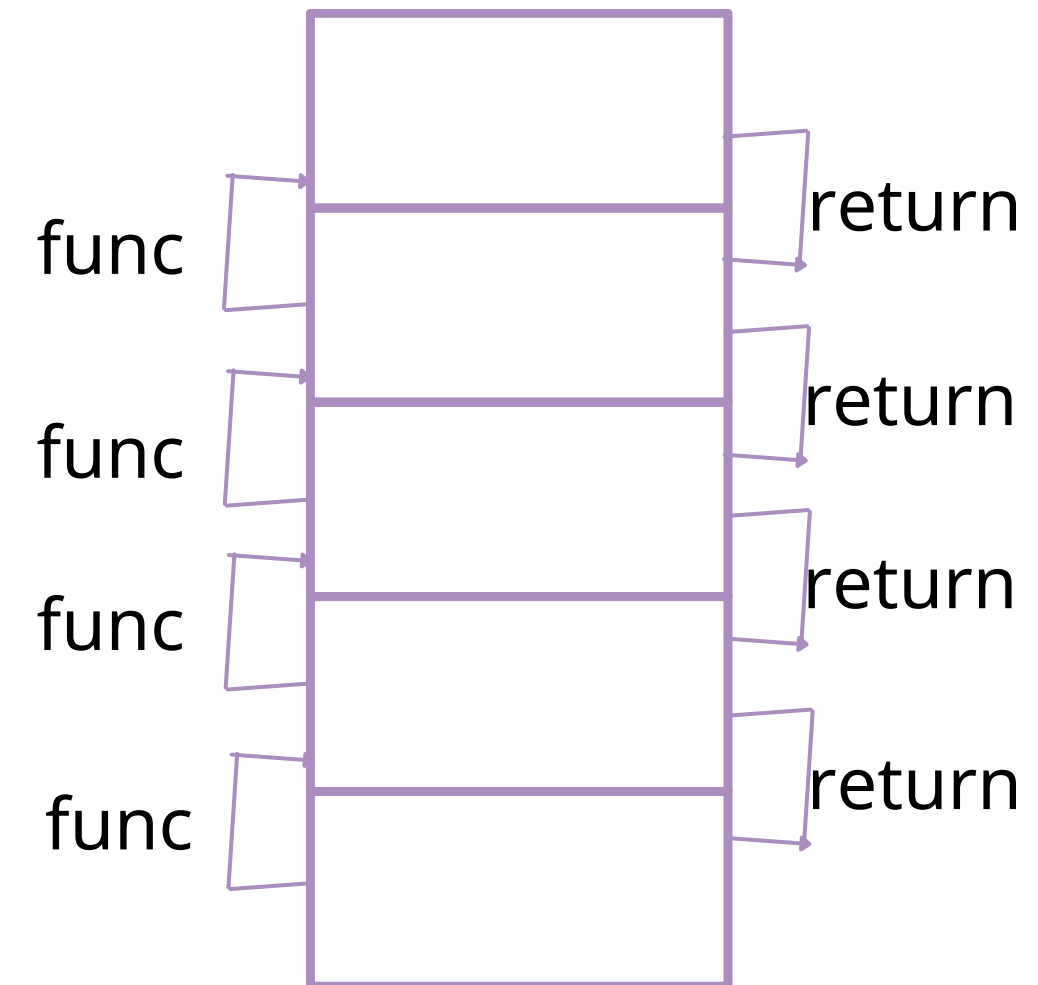
- 함수안에서 다시 재귀하여 함수 실행

```
1 def recursive(n):  
2     if n == 1:  
3         return 1  
4     return n * recursive(n-1)  
5  
6 print(recursive(3))
```



- 재귀에는 한도가 있다.

예외가 발생했습니다. **RecursionError** ×
maximum recursion depth exceeded while calling a Python object



Function of Python

재귀

- 피보나치 수열
- 안 쓰고 만들 수 있다면 그렇게 하자

```
def fibo(n):  
    if n == 0 or n == 1:  
        return 1  
    return fibo(n - 2) + fibo(n - 1)
```

```
def fibo_list(n):  
    for i in range(n):  
        print(fibo(i), end=" ")
```

```
fibo_list(10)
```

```
1 1 2 3 5 8 13 21 34 55
```

```
# 피보나치 수열 - 재귀함수 없이
```

```
def fibonacci(n):  
    f_num = [_ for _ in range(n)]  
    for i in range(n-2):  
        f_num[i + 2] = f_num[i] + f_num[i + 1]  
  
    return f_num
```

```
fibonacci(10)
```

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

“의논할 의”의 “의”라는 글자는 한자로 “의논할 의”

```
def countdown(n):  
    if n == 0:  
        print("Blastoff!")  
    else:  
        print(n)  
        countdown(n-1)
```

```
>>> countdown(3)  
3  
2  
1  
Blastoff!
```

```
def hello(count):  
    if count == 0:    # 종료 조건을 만듦.  
        return  
  
    print('Hello, world!', count)  
  
    count -= 1        # count를 1 감소시킨  
    hello(count)      # 다시 hello에 넣음
```

```
hello(5)    # hello 함수 호출
```

```
Hello, world! 5  
Hello, world! 4  
Hello, world! 3  
Hello, world! 2  
Hello, world! 1
```

```
def factorial(n):  
    if n == 1:        # n이 1일 때  
        return 1      # 1을 반환하고 재귀호출을 끝냄  
    return n * factorial(n - 1)    # n과 factorial 함수에 n - 1을 넣어서 반환된 값을 곱함  
  
print(factorial(5)) 120
```

```
def hello():  
    print('Hello, world!')  
    hello()
```

```
hello()
```

```
Hello, world!  
Hello, world!  
Hello, world!  
...(생략)
```

Traceback (most recent call last):

```
File "C:\project\recursive_function_error.py", line 5, in <module>  
    hello()  
File "C:\project\recursive_function_error.py", line 3, in hello  
    hello()  
File "C:\project\recursive_function_error.py", line 3, in hello  
    hello()  
File "C:\project\recursive_function_error.py", line 3, in hello  
    hello()  
[Previous line repeated 974 more times]
```

```
File "C:\project\recursive_function_error.py", line 2, in hello  
    print('Hello, world!')
```

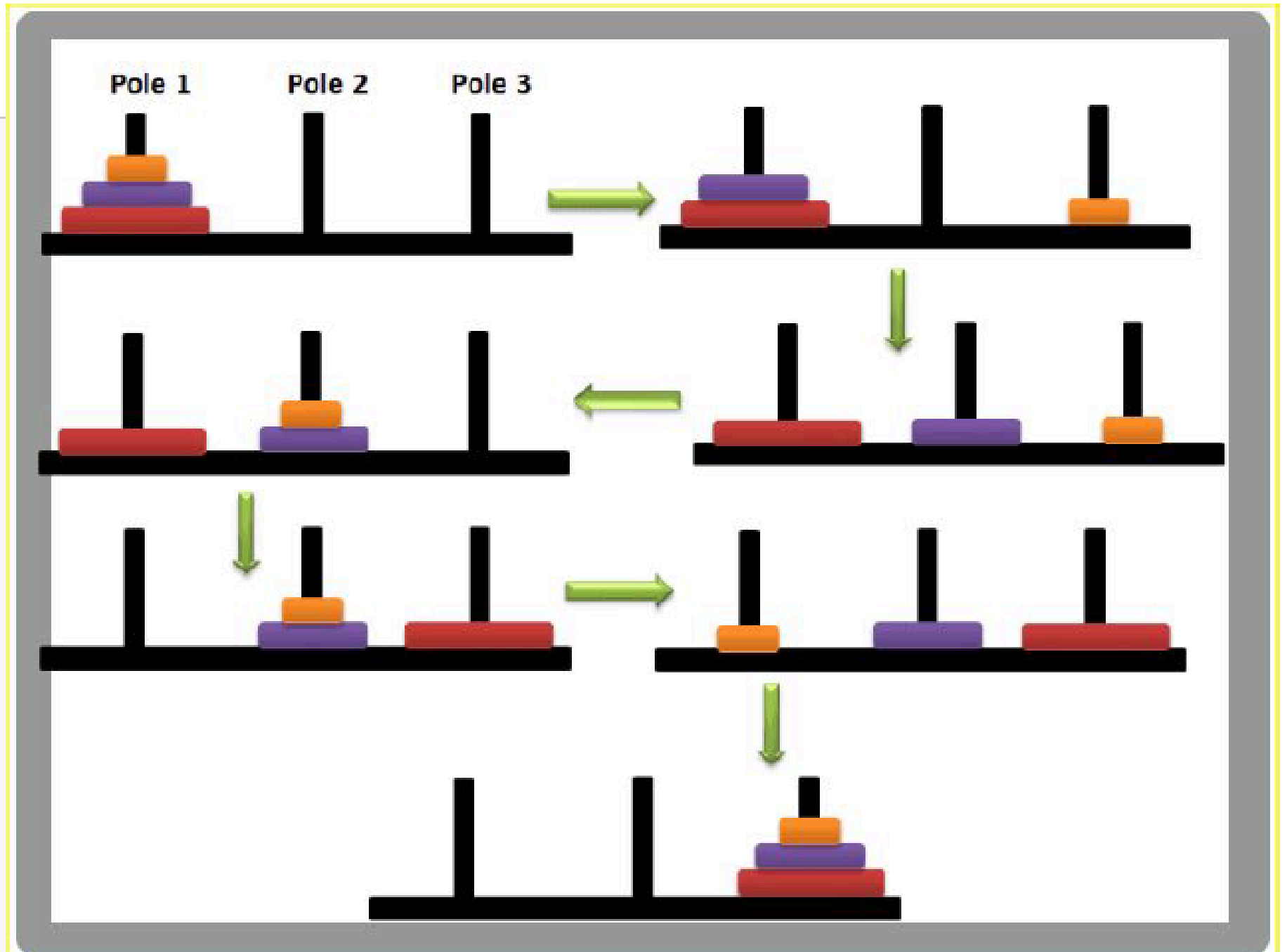
RecursionError: maximum recursion depth exceeded while pickling an object

하노이의 탑

- [참고](#)
- A에 있는 n개의 원반 중 맨 아래에 있는 n번째 원반을 제외한 나머지 원반을 모두 B로 옮긴다.
- A에 남은 하나의 원반을 C로 옮긴다.
- B의 모든 원반을 C로 옮긴다.

```
def hanoi(num, _from, _by, _to):  
    #탈출 조건  
    if num == 1:  
        print("{}에서 {}로 {} 번째 원반 이동".format(_from, _to, num))  
        return  
  
    hanoi(num-1, _from, _to, _by)  
    print("{}에서 {}로 {} 번째 원반 이동".format(_from, _to, num))  
    hanoi(num-1, _by, _from, _to)  
  
while 1:  
    numOfTray = int(input("원반의 개수를 입력하세요!(종료 : 0) :"))  
    if numOfTray == 0:  
        break  
    hanoi(numOfTray, 'A', 'B', 'C')
```

작은 원반 위에 큰 원반이 올 수 없다!
Hint: 두 개의 판을 옮기는 문제를 반복하는 것



불변객체-가변객체

call by reference in function

Function of Python

가변객체와 불변객체가 여기서 또?!

- 가변 객체(mutable)는 함수의 인자로 입력되었을 때 함수 내부에서 이루어진 변형이 외부에도 적용
- 불변 객체(immutable)은 해당x - “불변”!
- 가변 객체를 수정하지 않고 재할당 할 경우에는 외부에 영향x
 - 인자가 함수 내부에서 다른 주소의 값을 할당 받았기 때문
 - 따라서 변수 주소의 데이터는 변경되지 않는다.
 - deepcopy에 대해 떠올려 보자

```
def func(immutable, mutable):  
    immutable += 1  
    mutable.append(1)
```

```
a = 1  
b = [1]
```

```
func(a, b)  
print(a, b)
```

출력 1 [1, 1]

Function of Python

값에 의한 호출 vs 참조에 의한 호출

값에 의한 호출

변수를 복제하여 함수 안에서 사용

복제된 값이기 때문에 인자는 변화가 없음

변수가 차지하는 메모리 공간이 작은 경우 사용됨

참조에 의한 호출

변수가 함수로 전달되어 사용

전달된 값이기 때문에 함수 안에서
매개변수의 값이 변하면 인자도 변화함

변수가 차지하는 메모리 공간이 큰 경우
사용됨

Function of Python

값에 의한 호출

```
call_by_value = 1

def func(call_by_value):
    call_by_value += 1
    print("In function")
    print(f"value: {call_by_value}, id: {id(call_by_value)}")

print("main")
print(f"value: {call_by_value}, id: {id(call_by_value)}")
func(call_by_value)
print("main")
print(f"value: {call_by_value}, id: {id(call_by_value)}")
```

출력

```
main
value: 1, id: 140706297419424
In function
value: 2, id: 140706297419456
main
value: 1, id: 140706297419424
```

- 변수 안에 있는 값을 복사하여 사용하기 때문에 변수의 id가 다르다.
- 함수가 끝나도 인자의 값은 바뀌지 않는다.

Function of Python

참조에 의한 호출

```
call_by_ref = [1]

def func(call_by_ref):
    call_by_ref.append(1)
    print("In function")
    print(f"value: {call_by_ref}, id: {id(call_by_ref)}")

print("main")
print(f"value: {call_by_ref}, id: {id(call_by_ref)}")
func(call_by_ref)
print("main")
print(f"value: {call_by_ref}, id: {id(call_by_ref)}")
```

출력

```
main
value: [1], id: 1747301949056
In function
value: [1, 1], id: 1747301949056
main
value: [1, 1], id: 1747301949056
```

- 인자를 그대로 함수 안에서 사용하기 때문에 변수의 id가 같다.
- 함수가 끝나도 인자의 값이 변경된 상태 그대로

Function of Python

참조에 의한 호출 사용 이유

변수를 복제 한다는 것은 메모리를 사용한다는 것

정수를 담고 있는 길이가 100인 리스트를 상상해보는다면, 이 리스트 하나를 인자로 넘겨주는 것은 정수형 인자를 100개 넘기는 것과 비슷하다. 또한 리스트는 늘어나기도, 줄어들기도 한다.

메모리 관리 차원이나 실행속도에서 크기가 큰 자료형을 복제한다는 것은 비효율적.

→ 참조라는 개념이 필요

Function of Python

참조에 의한 호출 사용 이유

- 메모리는 주소와 값으로 이루어져 있다.
- 우리가 변수를 쓰는 것은 주소 x번지에서 값을 불러오는 것이다.
- 값에 의한 호출은 인자의 값을 받아서 매개변수에 복사해서 넣어주는 것
- 복사하는 값이 크다면, 비효율적
- 주소를 전달해주는 개념이 필요

주소	값
1	13123
2	"text"
3	[1, 2, 3, 4, 5]
4	object
5	1.42
6	["asdf", "text", 123]

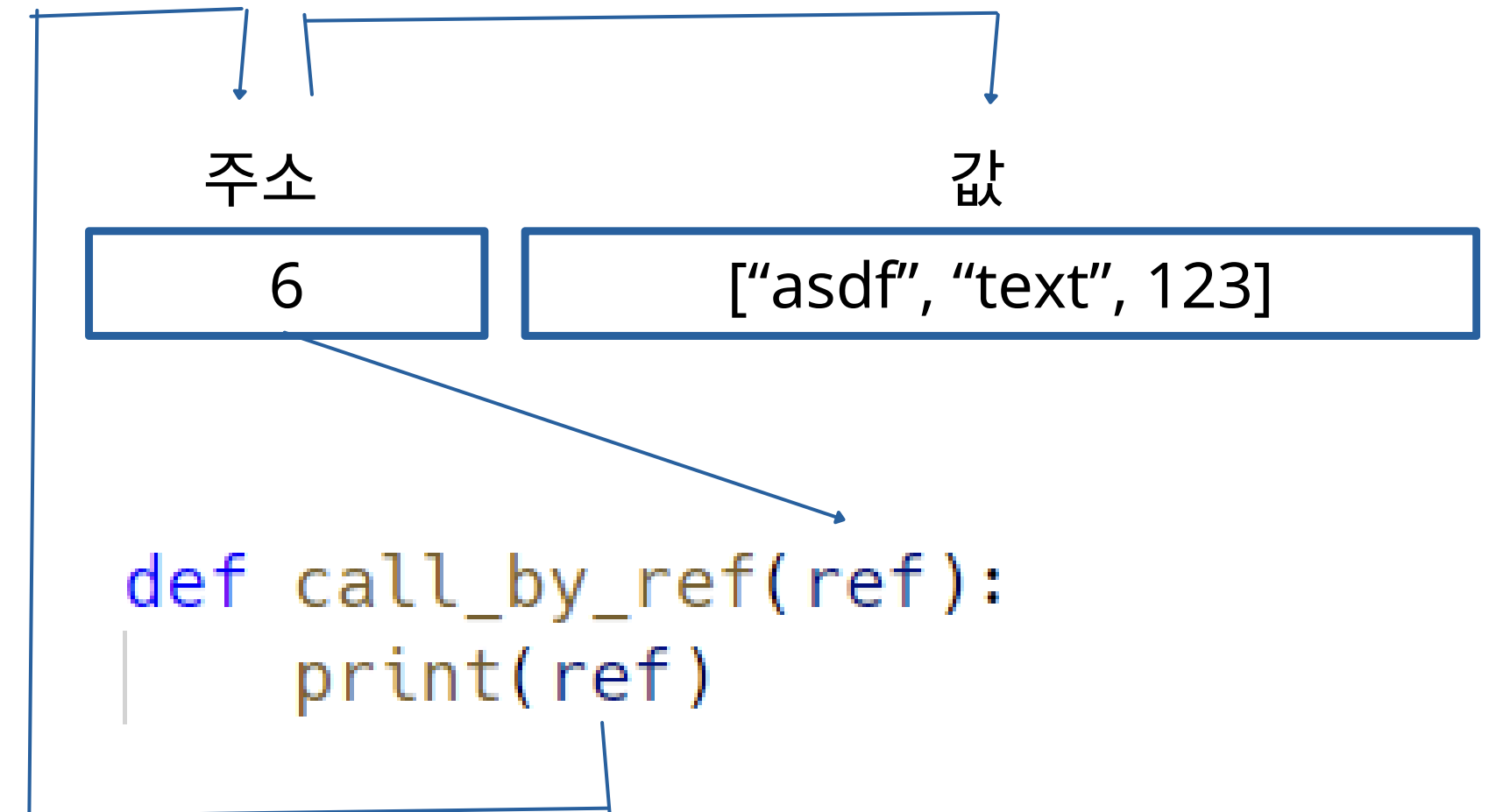
메모리에 할당되는 크기

이해를 돕기 위한 그림
똑같은 방식으로 동작하지는 않는다.

Function of Python

참조에 의한 호출 사용 이유

- 크기가 큰 자료형은 주소를 넘기고, 함수에서 그 주소를 이용해 호출
- 인자 자체를 쓰는 것과 동일하기 때문에 함수 밖으로 나왔을 때도 인자는 변경되어 있다.



built-in functions

기본 함수들

Built-in functions

Built-in Functions

A <u>abs()</u> <u>aiter()</u> <u>all()</u> <u>anext()</u> <u>any()</u> <u>ascii()</u>	E <u>enumerate()</u> <u>eval()</u> <u>exec()</u>	L <u>len()</u> <u>list()</u> <u>locals()</u>	R <u>range()</u> <u>repr()</u> <u>reversed()</u> <u>round()</u>
B <u>bin()</u> <u>bool()</u> <u>breakpoint()</u> <u>bytearray()</u> <u>bytes()</u>	F <u>filter()</u> <u>float()</u> <u>format()</u> <u>frozenset()</u>	M <u>map()</u> <u>max()</u> <u>memoryview()</u> <u>min()</u>	S <u>set()</u> <u>setattr()</u> <u>slice()</u> <u>sorted()</u> <u>staticmethod()</u> <u>str()</u> <u>sum()</u> <u>super()</u>
C <u>callable()</u> <u>chr()</u> <u>classmethod()</u> <u>compile()</u> <u>complex()</u>	G <u>getattr()</u> <u>globals()</u>	N <u>next()</u>	T <u>tuple()</u> <u>type()</u>
D <u>delattr()</u> <u>dict()</u> <u>dir()</u> <u>divmod()</u>	H <u>hasattr()</u> <u>hash()</u> <u>help()</u> <u>hex()</u>	O <u>object()</u> <u>oct()</u> <u>open()</u> <u>ord()</u>	V <u>vars()</u>
	I <u>id()</u> <u>input()</u> <u>int()</u> <u>isinstance()</u> <u>issubclass()</u> <u>iter()</u>	P <u>pow()</u> <u>print()</u> <u>property()</u>	Z <u>zip()</u>
			- <u>__import__()</u>

ascii, bin, bool, bytes, bytearray, chr, dict, frozenset, float, hex, int, list, object, oct, ord, set, tuple, str,

divmod, abs, sum, max, min, pow, round, all, any, len, iter, next , enumerate, range, zip

eval, exec, hash, map, filter, reversed, slice, sorted,

print, input, id, type, property, repr, setattr, getattr, isinstacne, issubclass, callable, globals, locals, vars, super, staticmethod

memoryview, breakpoint, aiter, anext,

Function of Python

내장함수

- import 없이 쓸 수 있는 함수들이다
- print, bool, help, len, locals, globals 등
- 모두 built-in function들

		내장 함수		
<code>abs()</code>	<code>delattr()</code>	<code>hash()</code>	<code>memoryview()</code>	<code>set()</code>
<code>all()</code>	<code>dict()</code>	<code>help()</code>	<code>min()</code>	<code>setattr()</code>
<code>any()</code>	<code>dir()</code>	<code>hex()</code>	<code>next()</code>	<code>slice()</code>
<code>ascii()</code>	<code>divmod()</code>	<code>id()</code>	<code>object()</code>	<code>sorted()</code>
<code>bin()</code>	<code>enumerate()</code>	<code>input()</code>	<code>oct()</code>	<code>staticmethod()</code>
<code>bool()</code>	<code>eval()</code>	<code>int()</code>	<code>open()</code>	<code>str()</code>
<code>breakpoint()</code>	<code>exec()</code>	<code>isinstance()</code>	<code>ord()</code>	<code>sum()</code>
<code>bytearray()</code>	<code>filter()</code>	<code>issubclass()</code>	<code>pow()</code>	<code>super()</code>
<code>bytes()</code>	<code>float()</code>	<code>iter()</code>	<code>print()</code>	<code>tuple()</code>
<code>callable()</code>	<code>format()</code>	<code>len()</code>	<code>property()</code>	<code>type()</code>
<code>chr()</code>	<code>frozenset()</code>	<code>list()</code>	<code>range()</code>	<code>vars()</code>
<code>classmethod()</code>	<code>getattr()</code>	<code>locals()</code>	<code>repr()</code>	<code>zip()</code>
<code>compile()</code>	<code>globals()</code>	<code>map()</code>	<code>reversed()</code>	<code>__import__()</code>
<code>complex()</code>	<code>hasattr()</code>	<code>max()</code>	<code>round()</code>	

Function of Python

내장함수

- divmod, abs, sum, max, min, pow, round

함수	설명	예시	결과
divmod	몫, 나머지	divmod(17, 5)	(3, 2)
abs	절댓값	abs(-7)	7
sum	합계	sum([1, 2, 3])	6
max	최댓값	max([1, 5, 2])	5
min	최솟값	min([1, 5, 2])	1
pow	거듭제곱	pow(2, 4)	16
round	반올림	round(3.456, 1)	3.5

```
# divmod(a, b): 몫과 나머지를 튜플로 반환
a, b = 17, 5
result = divmod(a, b)
print("divmod(17, 5) →", result)    # (3, 2)

# abs(x): 절댓값
print("abs(-10) →", abs(-10))      # 10
print("abs(3.14) →", abs(3.14))    # 3.14

# sum(iterable): 합계
numbers = [1, 2, 3, 4, 5]
print("sum([1,2,3,4,5]) →", sum(numbers))  # 15

# max(iterable): 최댓값
# min(iterable): 최솟값
nums = [10, 40, 5, 23]
print("max([10, 40, 5, 23]) →", max(nums))  # 40
print("min([10, 40, 5, 23]) →", min(nums))  # 5

# pow(x, y): x의 y제곱
# pow(x, y, z): (x의 y제곱) % z
print("pow(2, 3) →", pow(2, 3))            # 8
print("pow(2, 3, 5) →", pow(2, 3, 5))      # 3

# round(x[, ndigits]): 반올림
print("round(3.14159) →", round(3.14159))  # 3
print("round(3.14159, 2) →", round(3.14159, 2))  # 3.14
```

Function of Python

내장함수

- bin, hex, oct, bytes, bytearray,
- ascii, chr, ord
- bool, float, int, str,
- list, dict, set,
- tuple, frozenset,

진법 변환 함수들

```
print("bin(10) →", bin(10))      # 2진수: 0b1010
print("hex(255) →", hex(255))    # 16진수: 0xff
print("oct(8) →", oct(8))        # 8진수: 0o10
```

bytes(): 바이트 객체 생성 (불변)

```
b = bytes("hello", "utf-8")
print("bytes('hello', 'utf-8') →", b)  # b'hello'
```

bytearray(): 바이트 배열 생성 (가변)

```
ba = bytearray("hello", "utf-8")
print("bytearray('hello', 'utf-8') →", ba)  # bytearray(b'hello')
ba[0] = 72  # 'h' → 'H'
print("수정된 bytearray →", ba)
```

ascii(): 비ASCII 문자는 이스케이프 처리된 문자열 반환

```
print("ascii('안녕') →", ascii('안녕'))  # '\uc548\ub155'
```

chr(): 유니코드 숫자 → 문자

```
print("chr(65) →", chr(65))      # 'A'
print("chr(44032) →", chr(44032))  # '가'
```

ord(): 문자 → 유니코드 정수

```
print("ord('A') →", ord('A'))    # 65
print("ord('가') →", ord('가'))  # 44032
```

Function of Python

내장함수

- object: 파이썬 내장 함수 중 하나로, 최초의 빈 객체를 생성

```
o = object()
print(type(o)) # <class 'object'>
```

이 객체는 아무 속성도, 메서드도 직접 정의되어 있지 않음.
하지만 ==, !=, __str__, __repr__, __hash__ 등의 기본 동작을 가지고 있음

```
a = object()
b = object()

print(a == b)      # False (다른 객체니까)
print(a is b)      # False
print(hash(a))     # 객체 해시값 출력
print(a.__class__) # <class 'object'>
```

원래는 object를 상속하여 만들어졌으나,
Python 3부터는 적지 않음

```
class MyClass:
    pass

class MyClass(object):
    pass
```

Function of Python

내장함수

- all, any

함수	의미	반환값
<code>all(iterable)</code>	모든 요소가 참이면 True	모두 참 → True, 하나라도 거짓 → False
<code>any(iterable)</code>	하나라도 참이면 True	하나라도 참 → True, 전부 거짓 → False

```
values1 = [True, True, True]
values2 = [True, False, True]
values3 = [False, False, False]

# all() 예제
print("all(values1):", all(values1)) # True
print("all(values2):", all(values2)) # False

# any() 예제
print("any(values2):", any(values2)) # True
print("any(values3):", any(values3)) # False
```

```
data = ["apple", "banana", "", "grape"]

# 모두 비어있지 않은가?
print(all(data)) # False → ''는 False로 간주됨

# 하나라도 비어있는가?
print(any(not item for item in data)) # True

nums = [2, 4, 6, 8]

# 모든 수가 짝수인가?
print(all(n % 2 == 0 for n in nums)) # True

# 하나라도 5 이상인가?
print(any(n >= 5 for n in nums)) # True
```

Function of Python

내장함수

- eval, exec

함수	역할	반환값	주 용도
<code>eval()</code>	표현식(expression) 평가	결과 반환	계산식 평가, 사용자 입력 계산
<code>exec()</code>	문 전체(statement) 실행	반환값 없음	코드 실행, 함수/변수 정의

```
expr = "3 * (4 + 5)"
result = eval(expr)
print("eval 결과:", result)  # 27
```

```
code = """
def greet(name):
    print("Hello,", name)
"""

exec(code)
greet("Zinga")  # Hello, Zinga
```

Function of Python

globals, locals, vars, super

내장함수

- property, repr, setattr, getattr, isinstance, subclass, callable, staticmethod

```
print(callable(print))    # True (함수는 호출 가능)
print(callable(3))        # False (숫자는 호출 불가)

print(isinstance(42, int)) # True
print(isinstance("hi", (int, str))) # True (튜플로 다중 검사)
```

```
class Person:
    def __init__(self, name):
        self.name = name

    def __repr__(self):
        return f"Person(name='{self.name}')"

p = Person("Changmin")
print(repr(p)) # Person(name='Changmin')
```

```
class Math:
    @staticmethod
    def add(x, y):
        return x + y

print(Math.add(3, 4)) # 7
```

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def diameter(self):
        return self._radius * 2

c = Circle(5)
print(c.diameter) # 10
```

```
print(getattr(u, "name")) # Zinga
print(getattr(u, "age", 30)) # age가 없으면 기본값 30 반환
```

```
class Animal: pass
class Dog(Animal): pass

print(issubclass(Dog, Animal)) # True
print(issubclass(int, float)) # False
```

```
class User:
    pass

u = User()
setattr(u, "name", "Zinga")
print(u.name) # Zinga
```

Function of Python

내장함수

- vars, super

```
class User:
    def __init__(self, name):
        self.name = name
        self.level = 1
```

```
u = User("Zinga")
print(vars(u))
```

```
{'name': 'Zinga', 'level': 1}
```

```
class Animal:
    def speak(self):
        print("동물 소리!")
```

```
class Dog(Animal):
    def speak(self):
        super().speak()
        print("멍멍!")
```

```
d = Dog()
d.speak()
```

```
동물 소리!
멍멍!
```

Function of Python

내장함수

- globals, locals

```
x = 100

def show_globals():
    print("globals()['x'] =", globals()['x'])

show_globals()
```

```
globals()['x'] = 100
```

전역 네임스페이스 딕셔너리 반환
현재 모듈 수준(전역)의 변수들을 딕셔너리로 반환
globals()['변수명'] 형태로 접근하거나 수정 가능

```
def example():
    a = 10
    b = 20
    print("locals() =", locals())
```

```
example()
```

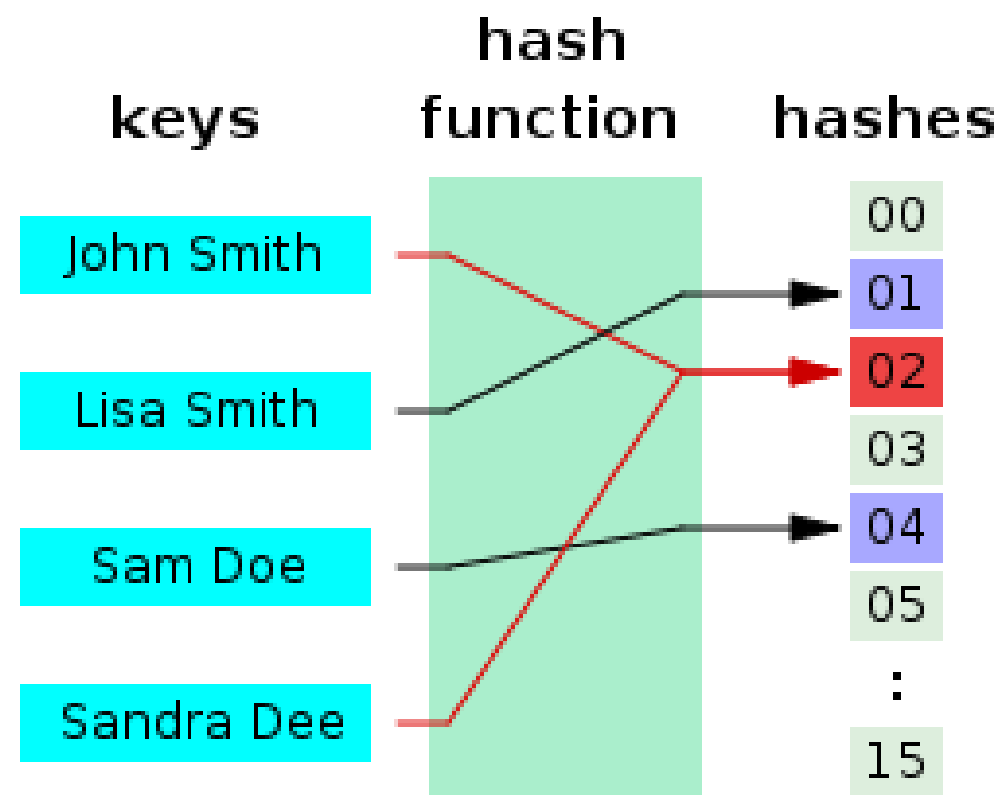
```
locals() = {'a': 10, 'b': 20}
```

현재 지역 네임스페이스 딕셔너리 반환
함수 내에서 호출하면 해당 지역 변수들의 딕셔너리 반환
전역에서 호출하면 globals()와 동일

Function of Python

내장함수

- hash,



길이가 다른 불변객체를
int값으로 변환해줌.
index처럼

```
# 정수, 실수, 문자열의 해시값
print("hash(42) →", hash(42))
print("hash(3.14) →", hash(3.14))
print("hash('hello') →", hash("hello"))
```

```
# 튜플 (mutable 객체는 안됨)
t = (1, 2, 3)
print("hash((1, 2, 3)) →", hash(t))
```

```
# 리스트는 불가능 (mutable)
try:
    hash([1, 2, 3])
except TypeError as e:
    print("리스트는 hash 불가:", e)
```

```
# 사용자 정의 클래스
```

```
class User:
    def __init__(self, name):
        self.name = name
```

```
u = User("Zinga")
print("기본 hash(User) →", hash(u)) # 메모리 주소 기반 해시
```

```
# __hash__ 오버라이드
```

```
class CustomUser:
    def __init__(self, name):
        self.name = name

    def __hash__(self):
        return hash(self.name)
```

```
cu = CustomUser("Zinga")
print("사용자 정의 hash →", hash(cu)) # name 기반 해시값
```

Function of Python

내장함수

- print, input, id, type,

이름: Zinga | 나이: 25

Zinga은 25살입니다.

id(x): 140703840323568

id(y): 140703840323568

id(z): 140703840323568

x, y, z는 같은 객체를 참조하나요? True

type(3): <class 'int'>

type(3.14): <class 'float'>

type('hello'): <class 'str'>

type([1, 2, 3]): <class 'list'>

```
# print(): 출력하기
name = "Zinga"
age = 25
print("이름:", name, "| 나이:", age)
print(f"{name}은 {age}살입니다.") # f-string 사용
```

```
print("-" * 40)
```

```
# input(): 사용자로부터 문자열 입력 받기
# (실습 시 주석 해제)
# user_input = input("당신의 취미는 무엇인가요? ")
# print("입력한 취미:", user_input)
```

```
print("-" * 40)
```

```
# id(): 객체의 고유 ID 확인 (메모리 주소와 유사)
```

```
x = 10
```

```
y = x
```

```
z = 10
```

```
print("id(x):", id(x))
```

```
print("id(y):", id(y))
```

```
print("id(z):", id(z))
```

```
print("x, y, z는 같은 객체를 참조하나요?", x is y and y is z)
```

```
print("-" * 40)
```

```
# type(): 자료형 확인
```

```
print("type(3):", type(3)) # int
```

```
print("type(3.14):", type(3.14)) # float
```

```
print("type('hello'):", type('hello')) # str
```

```
print("type([1, 2, 3]):", type([1, 2, 3])) # list
```

Function of Python

내장함수

- iter, next, len, enumerate, range, zip

```
# iter(): 반복 가능한 객체를 이터레이터로 변환
numbers = [1, 2, 3]
it = iter(numbers)
print("iter() 사용:", next(it)) # 1
print("다음 값:", next(it))      # 2
print("다음 값:", next(it))      # 3

print("-" * 40)

# next(): 이터레이터에서 다음 요소 가져오기
# 위에서 이미 사용됨 (next(it))

# len(): 요소 개수 반환
print("len(numbers):", len(numbers)) # 3
print("len('Python'):", len("Python")) # 6

print("-" * 40)
```

```
# next(): 이터레이터에서 다음 요소 가져오기
# 위에서 이미 사용됨 (next(it))

# len(): 요소 개수 반환
print("len(numbers):", len(numbers)) # 3
print("len('Python'):", len("Python")) # 6

print("-" * 40)

# enumerate(): 인덱스와 값을 함께 반환
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i}번 과일: {fruit}")

print("-" * 40)

# range(): 숫자 시퀀스 생성
for n in range(1, 6):
    print("range 값:", n)

print("-" * 40)

# zip(): 여러 시퀀스를 병렬로 묶기
names = ["Alice", "Bob", "Charlie"]
scores = [90, 85, 92]
for name, score in zip(names, scores):
    print(f"{name}의 점수는 {score}점입니다.")
```

Function of Python

내장함수

- map, filter, reversed, slice, sorted,

```
nums = [1, 2, 3, 4]
squares = list(map(lambda x: x**2, nums))
print("map → 제공된 리스트:", squares)  # [1, 4, 9, 16]

print("-" * 40)

# filter(): 조건을 만족하는 요소만 남김
evens = list(filter(lambda x: x % 2 == 0, nums))
print("filter → 짝수만 남기기:", evens)  # [2, 4]

print("-" * 40)

# reversed(): 요소 순서 뒤집기 (이터레이터 반환 → list로 변환해야 보임)
letters = ['a', 'b', 'c', 'd']
rev = list(reversed(letters))
print("reversed →", rev)  # ['d', 'c', 'b', 'a']

print("-" * 40)

# slice(): 시퀀스를 부분 추출하는 객체
s = slice(1, 4)  # 1번 인덱스부터 3번 인덱스까지 (4는 제외)
items = ['zero', 'one', 'two', 'three', 'four']
print("slice(1,4) →", items[s])  # ['one', 'two', 'three']

print("-" * 40)

# sorted(): 정렬된 새 리스트 반환 (원본은 변경되지 않음)
scores = [60, 95, 70, 85]
print("오름차순 정렬:", sorted(scores))  # [60, 70, 85, 95]
print("내림차순 정렬:", sorted(scores, reverse=True))  # [95, 85, 70, 60]
print("원본 유지:", scores)  # 원본은 그대로
```

Function of Python

내장함수

- enumerate – dict와 반복문과 함께

```
for i, k in enumerate(ages):  
    print(f"name : {i}, value: {k}")
```

```
name : 0, value: cm  
name : 1, value: hj  
name : 2, value: aa
```

```
for i, d in enumerate(ages.items()):  
    print(f"name : {i}, value: {d}")
```

```
name : 0, value: ('cm', 34)  
name : 1, value: ('hj', 31)  
name : 2, value: ('aa', 22)
```

```
for i, (d, k) in enumerate(ages.items()):  
    print(f"index: {i}, name: {d}, age: {k}")
```

```
index: 0, name: cm, age: 34  
index: 1, name: hj, age: 31  
index: 2, name: aa, age: 2
```

Function of Python

내장함수

- zip - 두 input의 길이가 같을 때 한 번에 for로 search

```
numbers = [1, 2, 3]
letters = ["A", "B", "C"]
for i in range(len(numbers)): 귀찮..
    print(f"{numbers[i]} {letters[i]}")

for i in zip(numbers, letters):
    print(f"{i}") 오..

for k,v in zip(numbers, letters):
    print(f"{k} {v}")
```

```
1 A
2 B
3 C
(1, 'A')
(2, 'B')
(3, 'C')
1 A
2 B
3 C
```

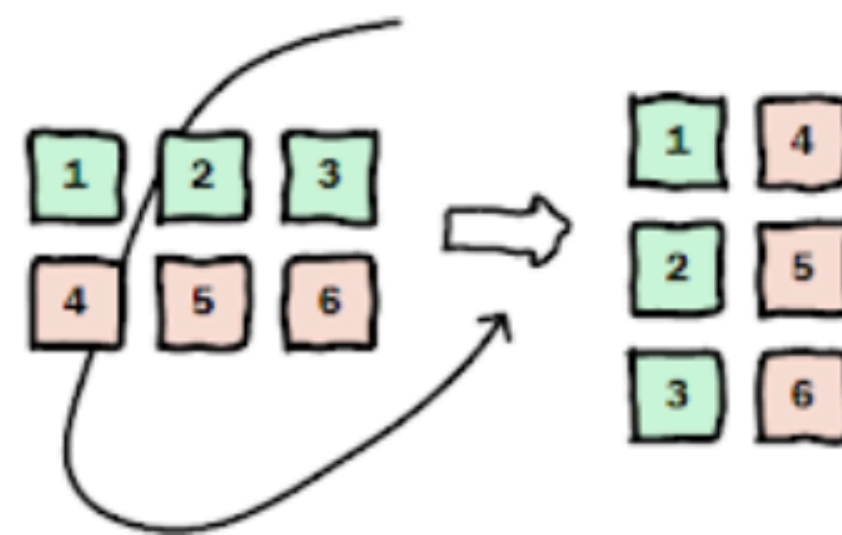
```
keys = [1, 2, 3]
values = ["A", "B", "C"]
dict(zip(keys, values))
```

```
{1: 'A', 2: 'B', 3: 'C'}
```

zip으로 dictionary 만들기!

주의! zip으로 묶는 인자들의 길이가 다르면 기본적으로 가장 짧은것을 기준으로 남긴다!!

활용예



```
A = [[1,2,3],[4,5,6]]
list(zip(*A))

[(1, 4), (2, 5), (3, 6)]
```

2d list transpose

Function of Python

내장함수

- sorted
 - list, tuple 등 iterable한 변수와 함께 사용
- 입력을 정렬(sort) 해줌
- 결과 값(return)은 list
- 기본적으로 오름차순 정렬
- Key를 통해 정렬 방식 커스텀 가능

```
help(sorted)
```

```
Help on built-in function sorted in module builtins:
```

```
sorted(iterable, /, *, key=None, reverse=False)
```

```
Return a new list containing all items from the iterable in ascending order.
```

```
A custom key function can be supplied to customize the sort order, and the  
reverse flag can be set to request the result in descending order.
```

```
my_input = [1,5,3,-5]  
sorted(my_input)
```

```
[-5, 1, 3, 5]
```

```
sorted(tuple(my_input))
```

```
[-5, 1, 3, 5]
```

```
my_input = [1,5,3,-5]  
sorted(my_input, reverse=True)
```

```
[5, 3, 1, -5]
```

```
def my_reverse(x): return 1/x  
sorted((1,5,3,-5), key = my_reverse)
```

```
[-5, 5, 3, 1]
```

```
sorted((1,5,3,-5), key = lambda x : 1/x)
```

```
[-5, 5, 3, 1]
```

람다는 편하지만 사용을 지양하자

Function of Python

내장함수

- Reversed
 - list, tuple 등 iterable한 변수와 함께 사용
- 입력을 내림차순 정렬(sort) 해줌
- Reverse iterator를 return한다.
- 이렇게 reverse 된 객체는 변수로 사용하려면 리스트나 튜플로 전환해줘야 함

```
list(reversed( ( 1, 2, 3 ) ))
```

```
[3, 2, 1]
```

```
for x in reversed( ( 1, 2, 3 ) ):
    print(x)
```

```
3
2
1
```

```
help(reversed)
```

Help on class reversed in module builtins:

```
class reversed(object)
|   reversed(sequence, /)
|
|   Return a reverse iterator over the values of the given sequence.
|
|   Methods defined here:
|
|   __getattr__(self, name, /)
|       Return getattr(self, name).
|
|   __iter__(self, /)
|       Implement iter(self).
|
|   __length_hint__(...)
|       Private method returning an estimate of len(list(it)).
|
|   __next__(self, /)
|       Implement next(self).
|
|   __reduce__(...)
|       Return state information for pickling.
|
|   __setstate__(...)
|       Set state information for unpickling.
|
|   -----
|   Static methods defined here:
|
|   __new__(*args, **kwargs) from builtins.type
|       Create and return a new object.  See help(type) for accurate signature.
```

lambda, map, filter와 list comprehension

어지간하면 list comprehension

```
def my_filter(x): return x>0 #True
my_lambda_filter = lambda x: x>0 #False
```

```
my_list = [-1,1,2,3,-4]

print(list(filter(my_filter, my_list)))
print(list(filter(my_lambda_filter, my_list)))

[1, 2, 3] 그 filter 결과를 다시 list로 변환
[1, 2, 3]
```

filter: iterable 항목을 하나씩 function에 넘겨 return이 true인 경우만 모음

```
print(list(map(my_filter, my_list)))

[False, True, True, True, False]
```

map: iterable 항목을 하나씩 function에 넘겨 return을 모아줌

```
[x for x in range(10)]
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
['-' for x in range(10)]
```

```
['-', '-', '-', '-', '-', '-', '-', '-', '-', '-']
```

```
[2*x for x in range(-5,11,2) if x>0]
```

```
[2, 6, 10, 14, 18]
```

```
[(x, y) for x in [1,2,3] for y in [-1,-2]]
```

```
[(1, -1), (1, -2), (2, -1), (2, -2), (3, -1), (3, -2)]
```

```
[x for x in range(10) if x < 5 if x % 2 == 0]
```

```
[0, 2, 4]
```

반복과 조건을 나열하여 리스트를 생성함!

이 방법이 훨씬 중요함!!!!

단, 조건이나 반복을 2개 이상 쓸거면 차라리 함수로 처리하세요

Function of Python

예시

- 파이썬 공식 문서 print 함수(<https://docs.python.org/3/library/functions.html#print>)
- Print(*Objects, sep=' ', end='\n', file=sys.stdout, flush=False)

```
1  f = open("test.txt", "w")
2  print("test1", file=f, end="\n")
3  print("test2", file=f, end="\n")
4
5  f = open('test.txt', 'r')
6  ✓ for line in f.readlines():
7    |     print(line, end="")
```

generator

lazy~메모리 효율적인 반복 처리 방식

generator

lazy return

정의:

- 값을 하나씩 순차적으로 생성하는 **반복자(iterator)**의 일종
- yield 키워드를 사용하여 값을 반환함

특징:

- 전체 값을 한 번에 메모리에 올리지 않고, 필요할 때마다 한 개씩 생성함
→ 메모리 효율성 극대화
- return과 비슷하지만, 값을 반환한 후 함수 상태를 저장하고 중단
- 다음 next() 호출 시 이전 상태부터 이어서 실행
- 메모리 절약: 큰 데이터를 한 번에 다루지 않고 필요한 순간에 하나씩 처리
- 무한 시퀀스도 구현 가능: count(), fibonacci()
- for 루프에서 자동으로 next() 호출

```
# 일반 함수
def normal():
    return [1, 2, 3]
```

```
# 제너레이터 함수
def gen():
    yield 1
    yield 2
    yield 3
```

```
g = gen()
print(next(g)) # 1
print(next(g)) # 2
```

Range를 만들어보자

```
def my_range(n):  
    i = 0  
    while i < n:  
        yield i  
        i += 1  
  
for num in my_range(5):  
    print(num)
```

0
1
2
3
4

generator expression

```
g = (x ** 2 for x in range(5))  
print(next(g))  # 0  
print(list(g))  # [1, 4, 9, 16]
```

map-filter를 lazy하게~

```
nums = range(1000000)  
squares = (x ** 2 for x in nums if x % 2 == 0)  
  
for i in squares:  
    if i > 100:  
        break  
    print(i)
```

lambda

익명함수

lambda function

익명함수

일반 함수

```
def add(x, y):  
    return x + y
```

람다 함수

```
add = lambda x, y: x + y
```

```
print(add(3, 5)) # 8
```

정상적인 함수 선언

```
square = lambda x: x ** 2  
print(square(4)) # 16
```

간단한 수식

```
is_even = lambda x: "Even" if x % 2 == 0 else "Odd"  
print(is_even(7)) # Odd
```

조건문을 포함

```
pairs = [(1, 3), (2, 2), (3, 1)]
sorted_pairs = sorted(pairs, key=lambda x: x[1])
print(sorted_pairs)
```

함수에 전달



```
def key_func(x):
    return x[1]
```

일반 함수명으로 넣어도됨..

✓
0초

```
[18] 1 nums = [1, 2, 3, 4, 5]
      2 squared = list(map(lambda x: x**2, nums))
      3 print(squared)
      4 evens = list(filter(lambda x: x % 2 == 0, nums))
      5 print(evens)
```

```
⇒ [1, 4, 9, 16, 25]
   [2, 4]
```

map,filter예제에 많음..